

Carnegie Mellon University

Data Analysis and Machine Learning

Computational Thinking

Tidy Data

Agenda

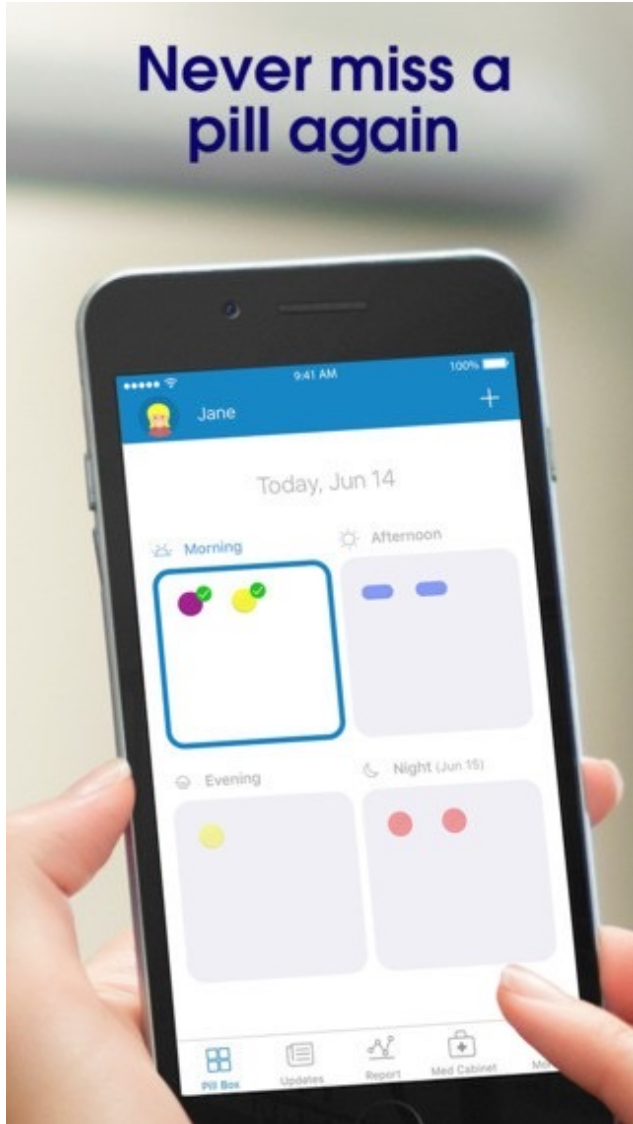
- Four facets of Computational Thinking
- Examples of abstraction
- Tidy data: observations and variables
- Mapping variables to visual elements

Take aways from last week

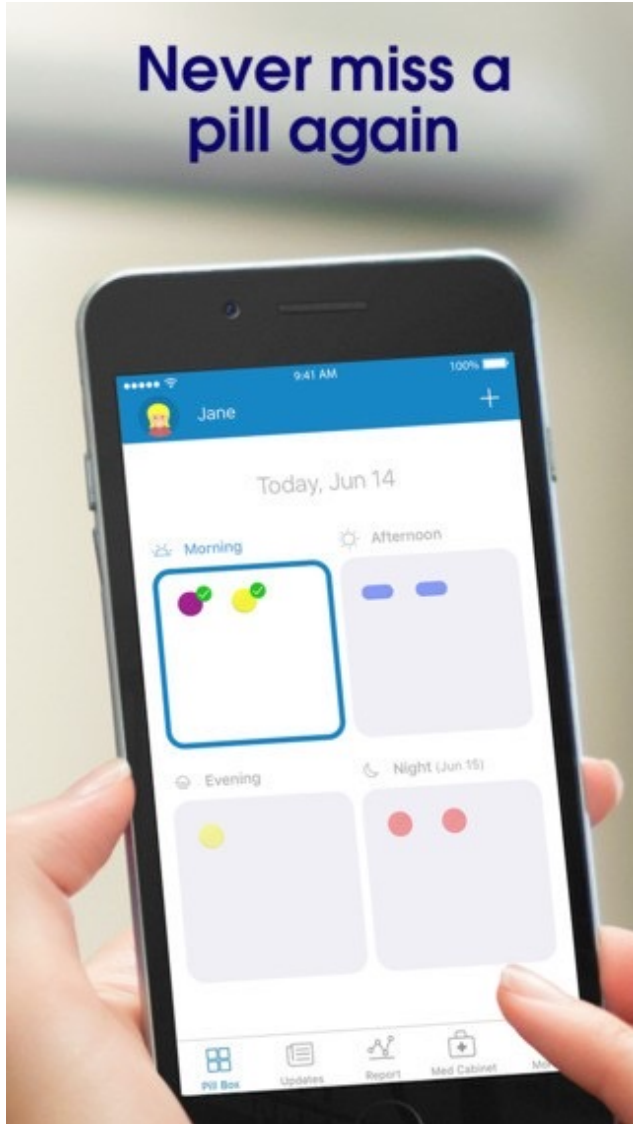
- Embrace your identity; know your heritage
- Course theme: Adding VALUE with data
- Four types of analytics:
 - descriptive: what happened?
 - diagnostic: why did it happen?
 - predictive: what could happen?
 - prescriptive: what should happen?
- Glimpse of:
 - Exploratory Data Analysis (visualization)
 - Machine Learning
 - Big Data

VALUE PROPOSITION

Solve the "problem" ... (it is NOT about technology)

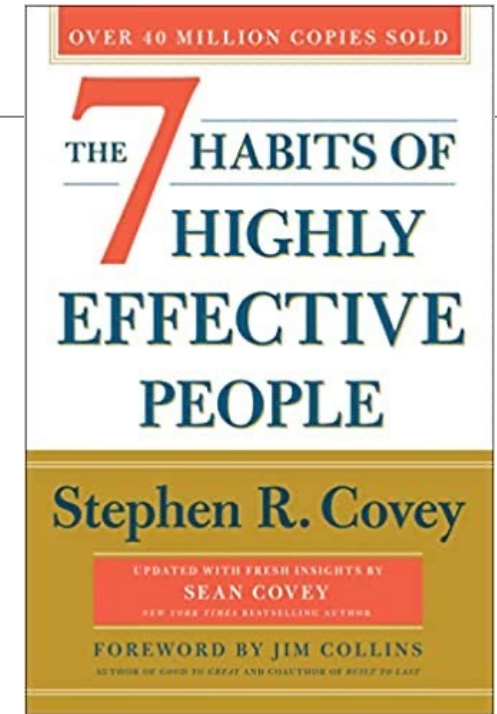
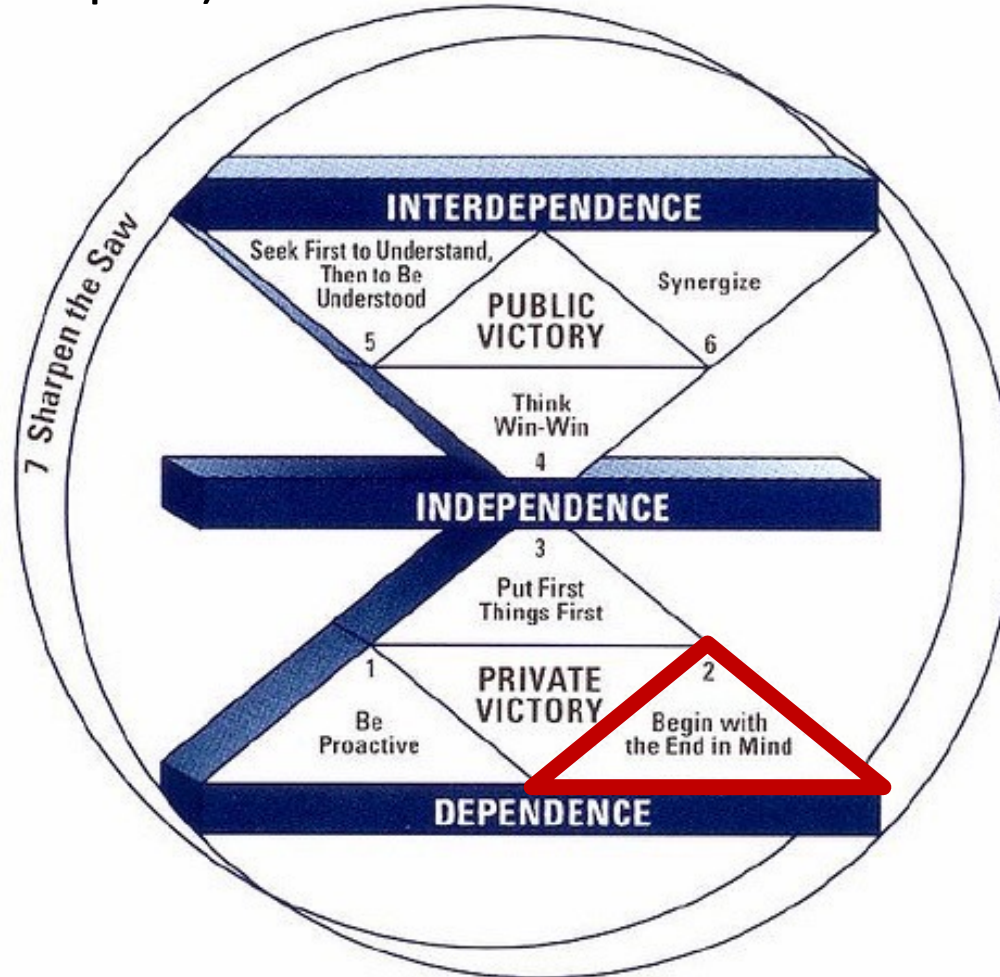


Solve the "problem" ... (it is NOT about technology)



Value Proposition (end-to-end)

- Pain Point
- Problem Formulation (measure the pain)
- Solution Development
- Deployment
- Evaluation (reduction in pain)
- Maintenance / Sustainability



Business Problem



Business Solution



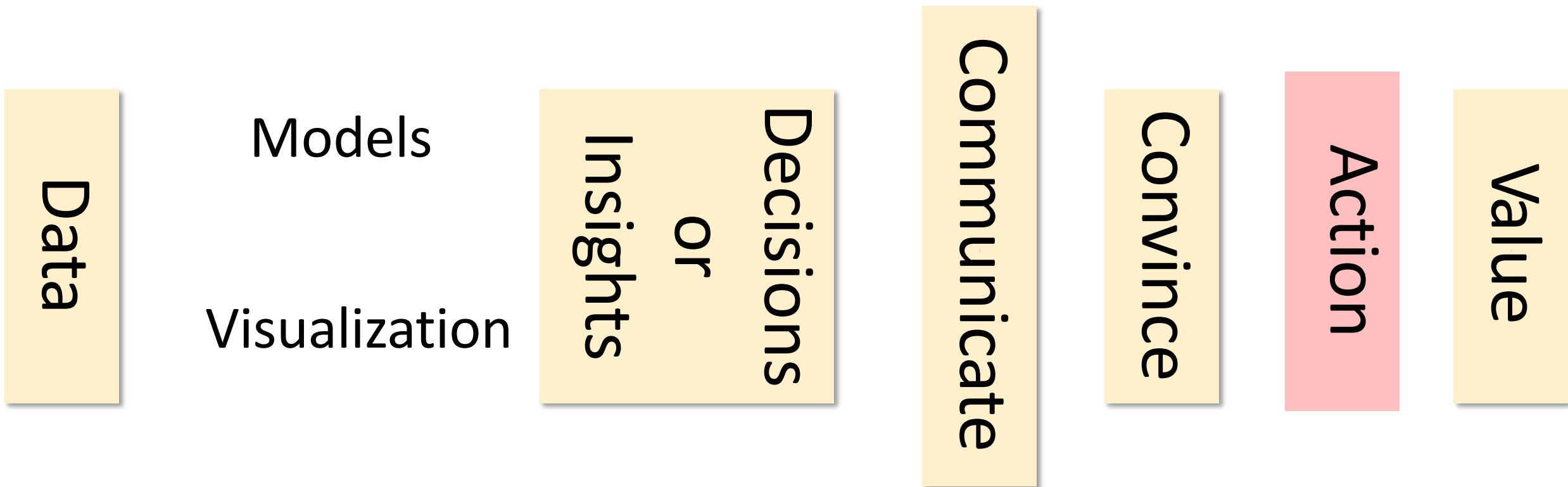
DA/ML Problem



DA/ML Solution



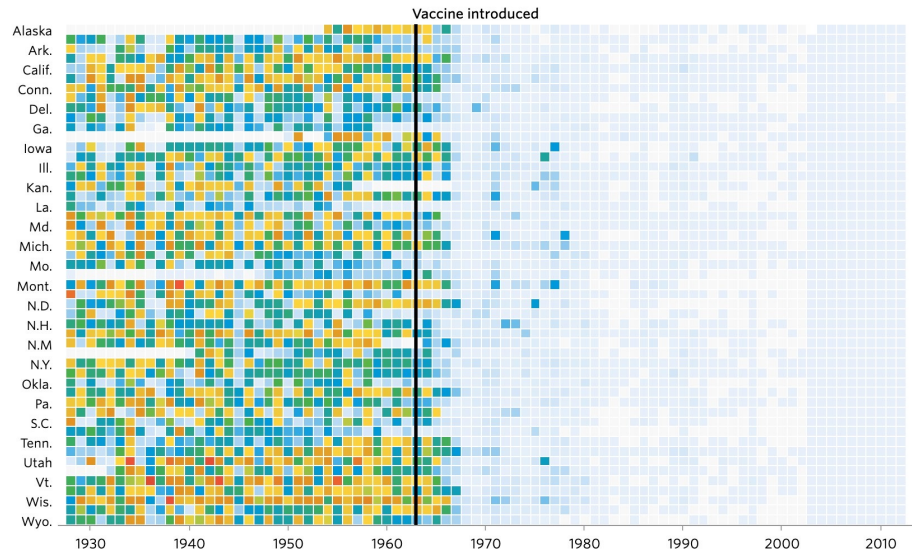
Analytics to Action



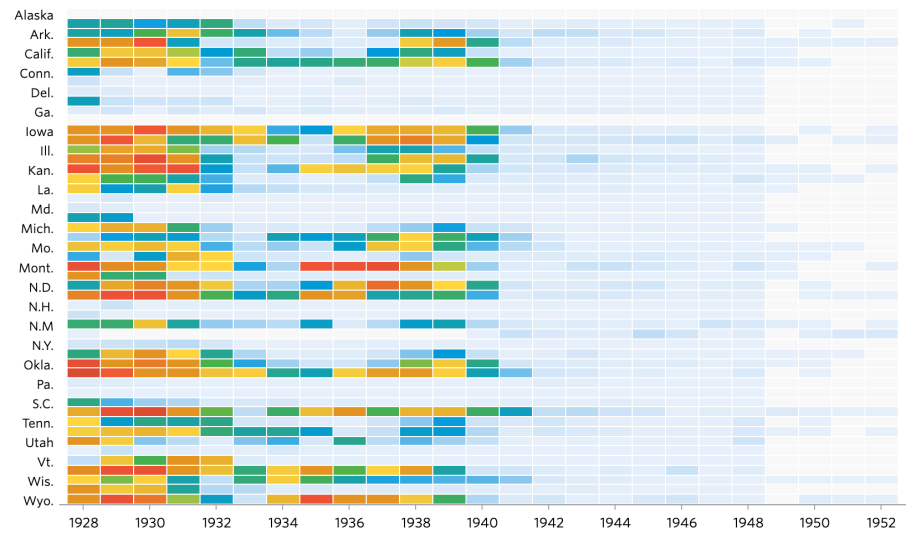
Battling Infectious Diseases in the 20th Century: The Impact of Vaccines

<http://graphics.wsj.com/infectious-diseases-and-vaccines/>

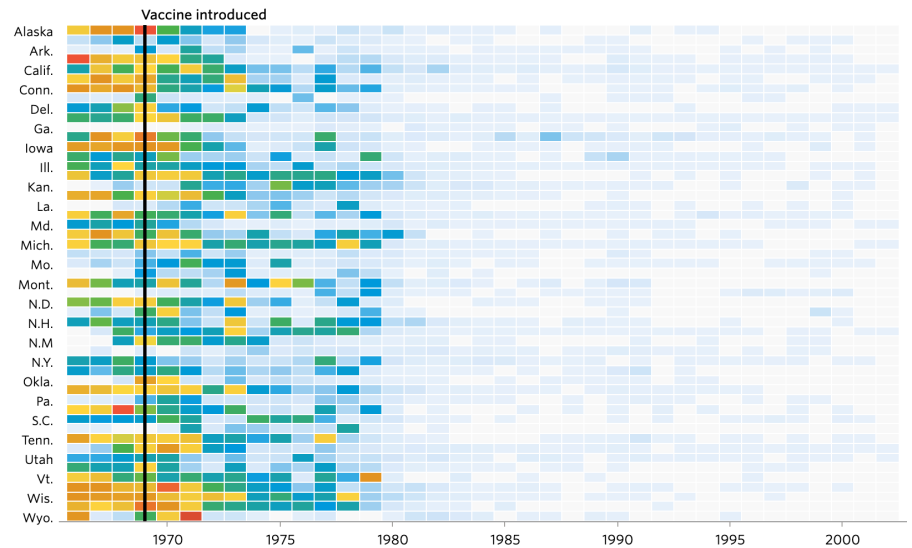
Measles



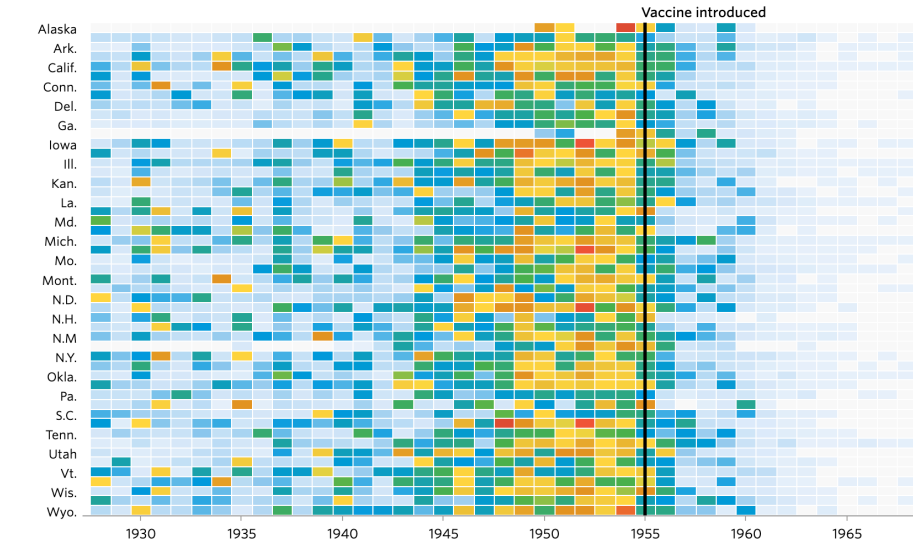
Smallpox



Rubella



Polio



COMPUTATIONAL THINKING

Decomposition

Abstraction

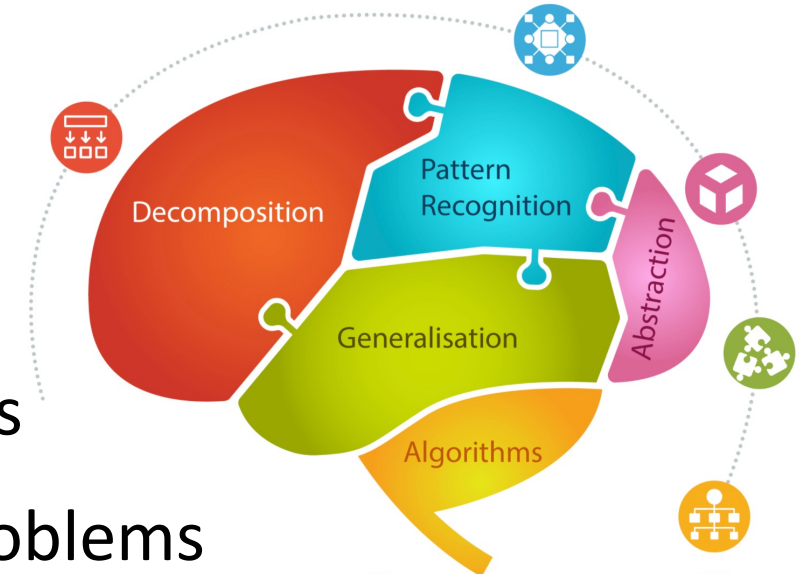
Recognition

Generalization

Computation

Components of Computational Thinking

- **Decomposition**: Divide and Conquer
- **Abstraction**: Separate the “what” from the “how”
- **Recognition**: Look for similarities between problems
- **Generalization**: Adapt previous solutions to new problems
- **Computation**: How to express steps unambiguously



Decomposition: Divide and Conquer



... one dish at a time ...

Being abstract is something profoundly different
from being vague... The purpose of abstraction is
not to be vague, but to create a *new semantic level*
in which one can be absolutely precise.
-- Edsger Dijkstra (1972 Turing award)



1930—2002

ABSTRACTION

The Mailbox Puzzle

There are a hundred mail boxes and they all are initially closed. A monkey goes in steps of 1,2,3,...,100 and opens all the mailboxes. The monkey comes back to the beginning and in steps of 2,4,6,...,100 closes the mailboxes. Then in steps of 3,6,9,...,99 if a mailbox is open it closes and if it is closed the monkey opens it. (In addition to being hyperactive the monkey is also smart and can count :-)). In this way the monkey goes in steps of 4,5,...,100 i.e., when going in steps of 50 it goes to the 50th and the 100th mailbox and if open closes it or vice-versa. After the monkey finishes this process, which mailboxes are open in the end?

The Mailbox Puzzle

There are a hundred mail boxes and they all are initially closed. A monkey goes in steps of 1,2,3,...,100 and opens all the mailboxes. The monkey comes back to the beginning and in steps of 2,4,6,...,100 closes the mailboxes. Then in steps of 3,6,9,...,99 if a mailbox is open it closes and if it is closed the monkey opens it. (In addition to being hyperactive the monkey is also smart and can count :-)). In this way the monkey goes in steps of 4,5,...,100 i.e., when going in steps of 50 it goes to the 50th and the 100th mailbox and if open closes it or vice-versa. After the monkey finishes this process, which mailboxes are open in the end?

This is the approach of *stratified design*, the notion that a complex system should be structured as a sequence of levels that are described using a sequence of languages. Each level is constructed by combining parts that are regarded as primitive at that level, and the parts constructed at each level are used as primitives at the next level. The language used at each level of a stratified design has *primitives*, means of *combination*, and means of *abstraction* appropriate to that level of detail.

- Abelson and Sussman

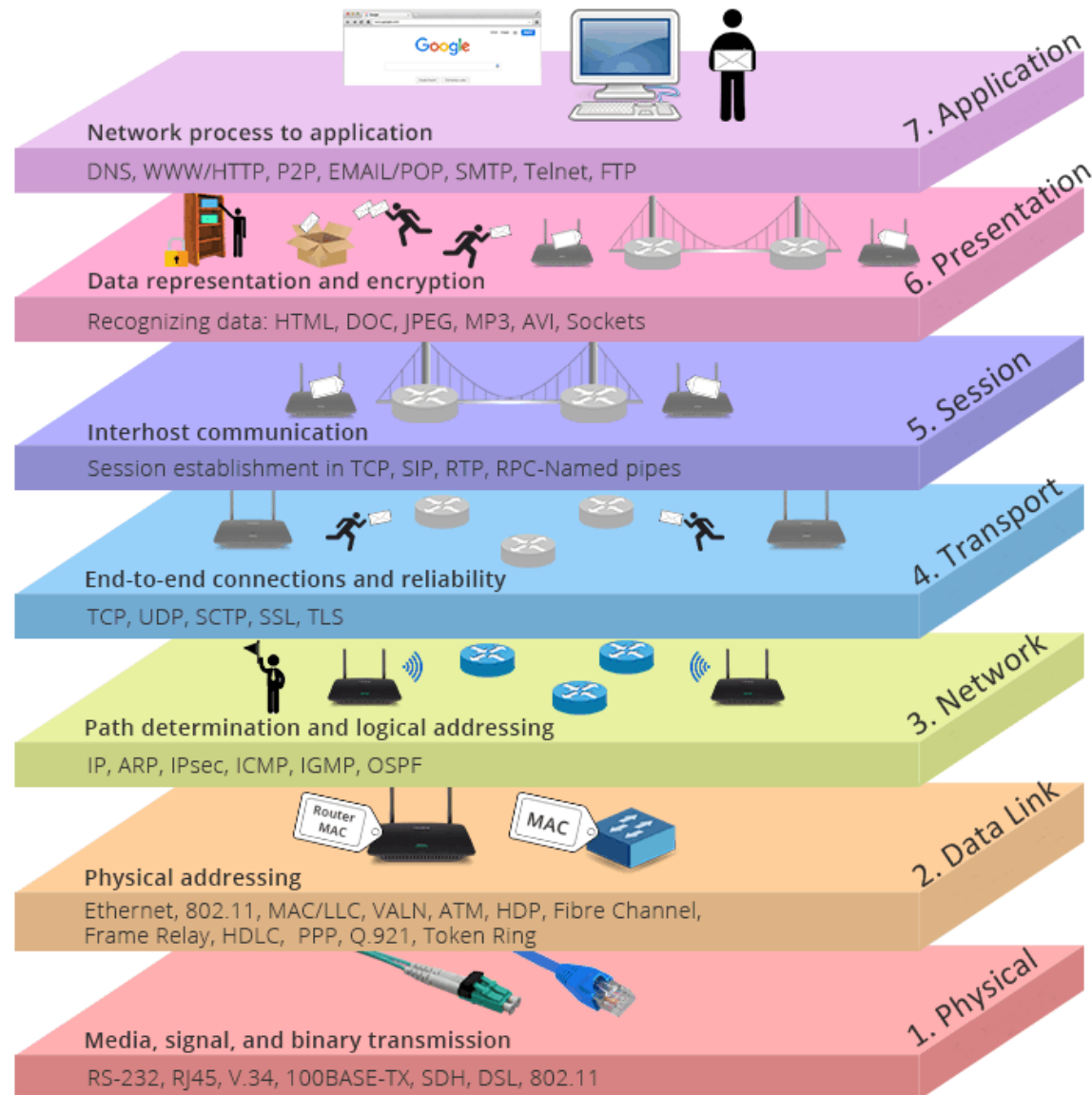
```
mailbox_puzzle
~~~~~
toggle toggle_all
~~~~~
open is_open close
~~~~~ <-- an abstraction barrier
representation of mailboxes
```

```
for i in range(1, 1+100):
    close(mbox, i)

for step in range(1, 1+100):
    toggle_all(mbox, step)

for i in range(1, 1+100):
    if is_open(mbox, i):
        print(i)
```

Examples of layered architectures: The Internet



Examples of layered architectures: Git

Git Commands

- ≈ 110 commands (plus options)
- Porcelain (user-facing)
 - 50 commands
 - CLI completion
- Plumbing ("core Git")
 - ≈ 60 commands
 - No CLI completion



Abstraction: Denoting computation

“sum of all integers from m to n”

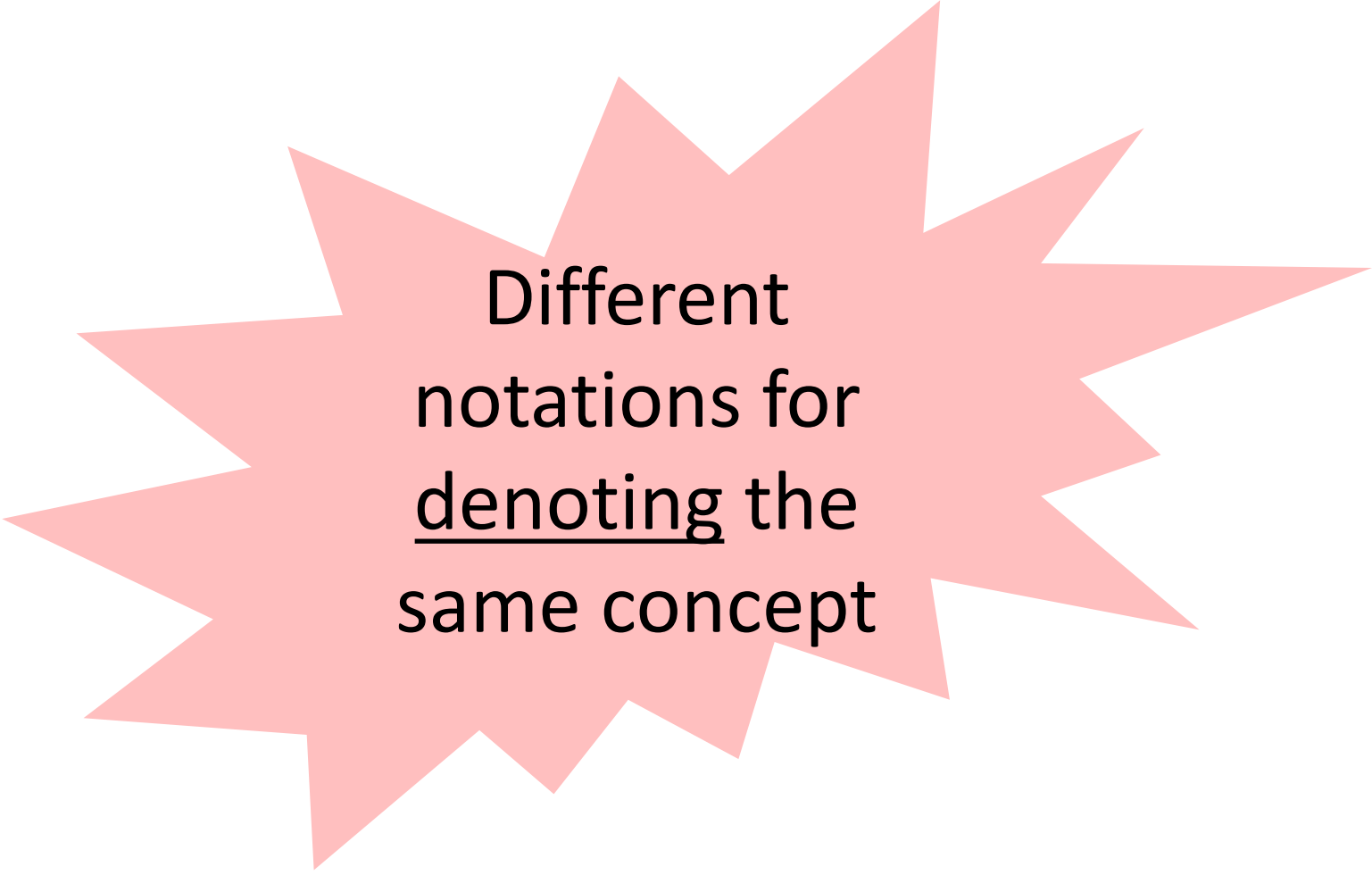
How: $m + (m+1) + \dots + (n-1) + n$

What:

■ Symbol $\sum_{i=m}^n i$ $\text{sigma}_{i=m}^n i$

■ Function $\text{sigma}(m, n)$

■ Message $m.\text{sigma}(n)$



Different
notations for
denoting the
same concept

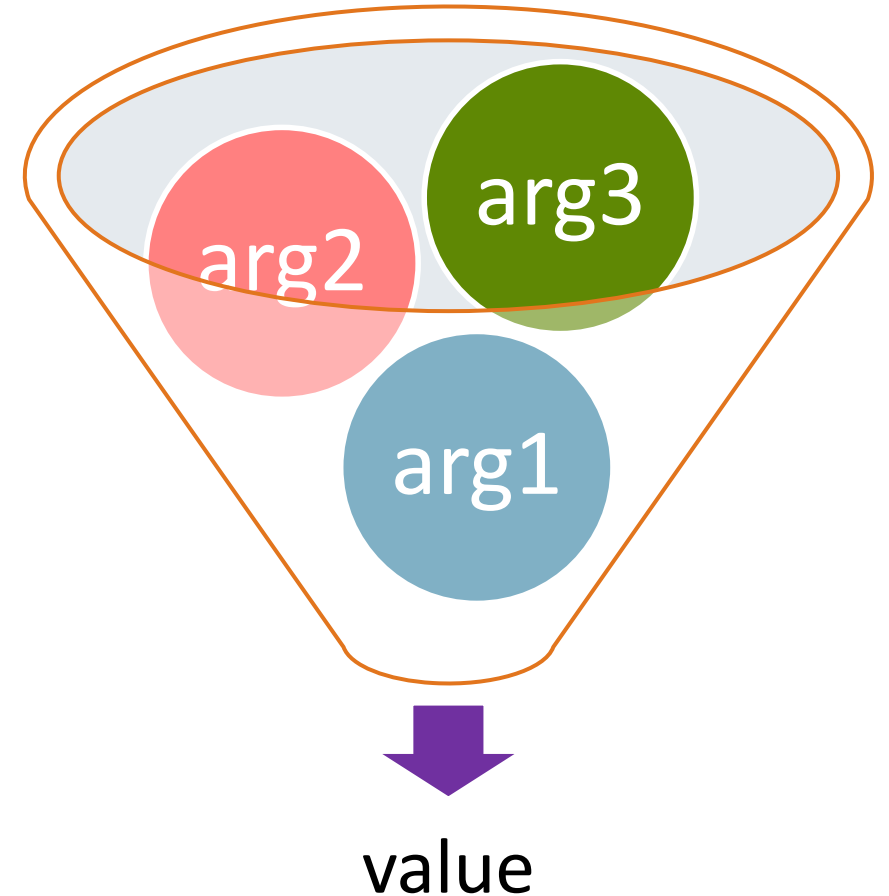
Concept of a function

```
x <- f( arg1, arg2, arg3 )
```

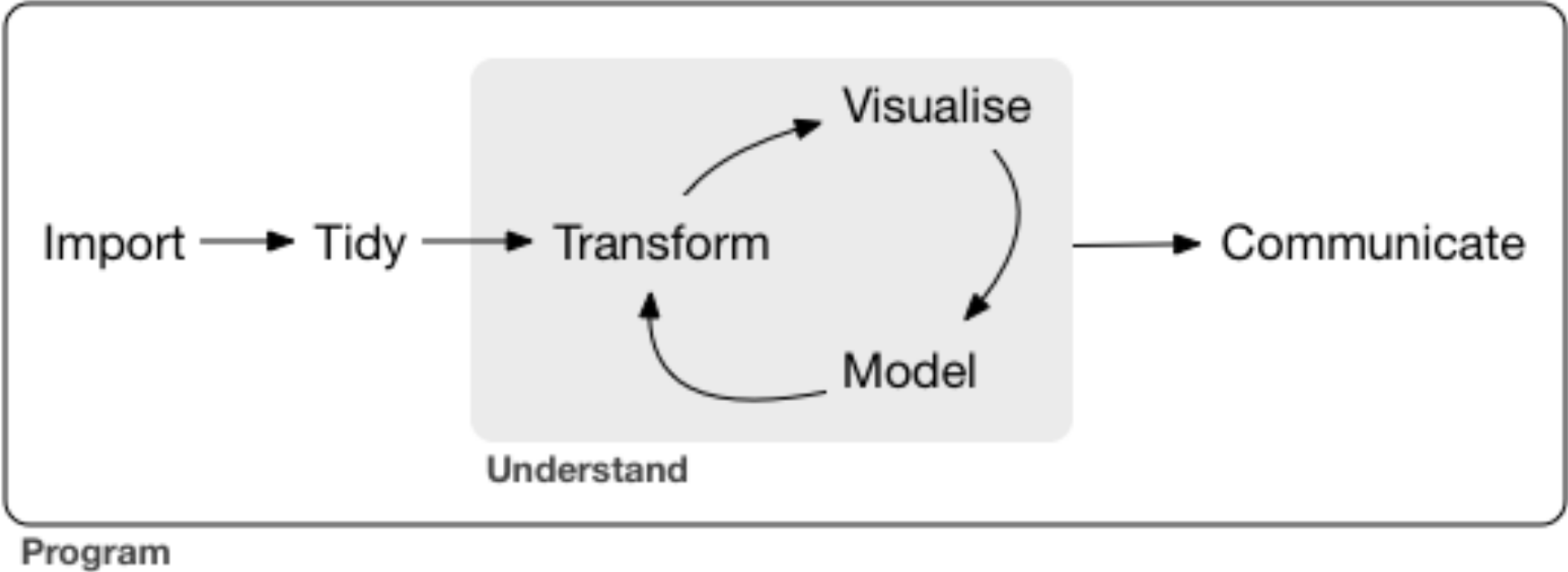
The number of arguments that are passed to a function is known as the function's **arity**

Functions process the **values** of its arguments and **return** 1 value

The name of the function and the arguments it takes tells us **what** the function does but not how it does it



Data



Value

Python Style Guide

Spring 2025, Data Science

Raja Sooriamurthi

Given the stage you are in your education, in addition to ensuring your code works, we also want to ensure that it conforms to the *pythonic* way of expressing a solution. The following guidelines on **abstraction**, **docstrings**, **destructuring assignments**, **list comprehensions**, **lambda expressions**, **line width** etc. should be followed in your code when applicable. Credit will be deducted for non-conformance and missed opportunities for better expressing your solutions.

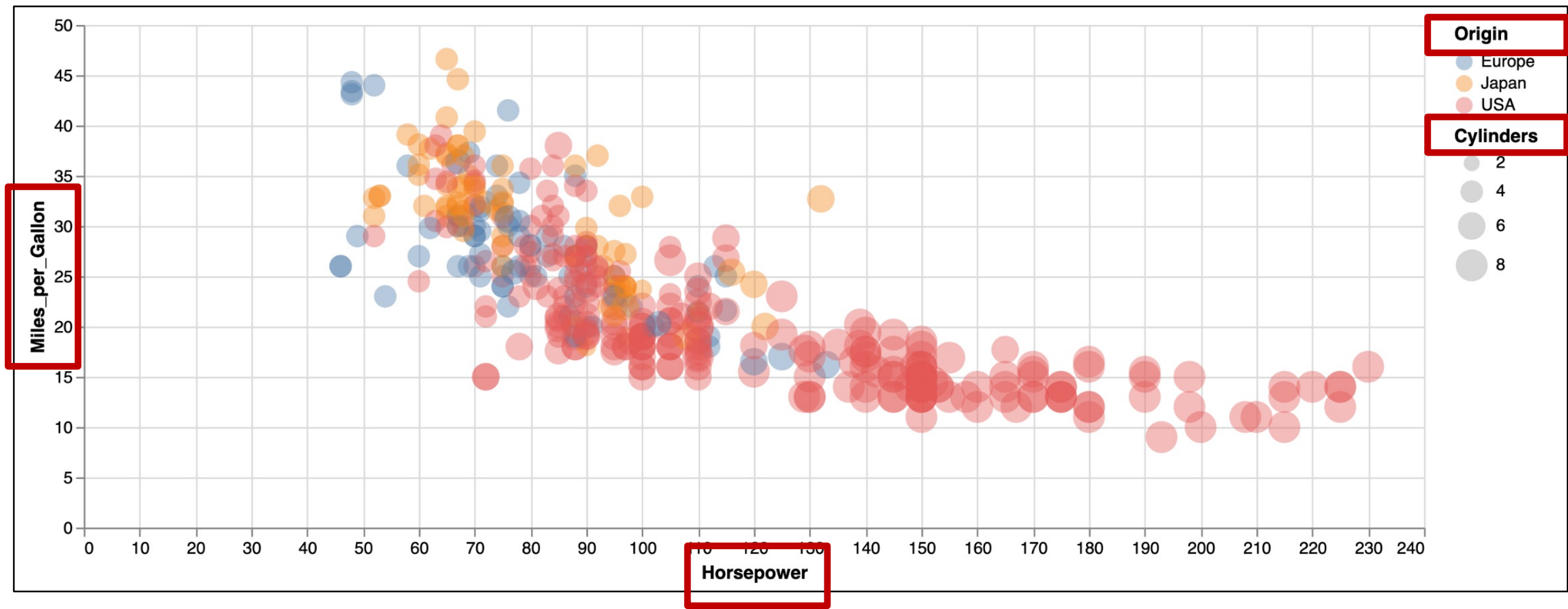
Practice making these guidelines an integral part of your thought process. Do not initially "think in Java" (or C) and say to yourself "... after I get this to work, I will clean up the code and make it more pythonic ...". While refactoring code is an excellent practice, do try to express your code the pythonic way from the beginning. Take this class as an opportunity to hone your Python skills.

- ✓ **Abstraction**: separate what from how (functional and data)
- ✓ **DocStrings**: what the functions does and how to call it
- ✓ **Destructuring assignments**: `a, *b = [1,2,3,4]`
- ✓ **List Comprehensions**: `[x**2 for x in [1,2,3,4,5,6] if x%2==0]`
- ✓ **Lambda expressions**: `lambda x: x[1]`
- ✓ **Line width**: no horizontal scrolling (~70 chars)

EFFECTIVE VISUALIZATION

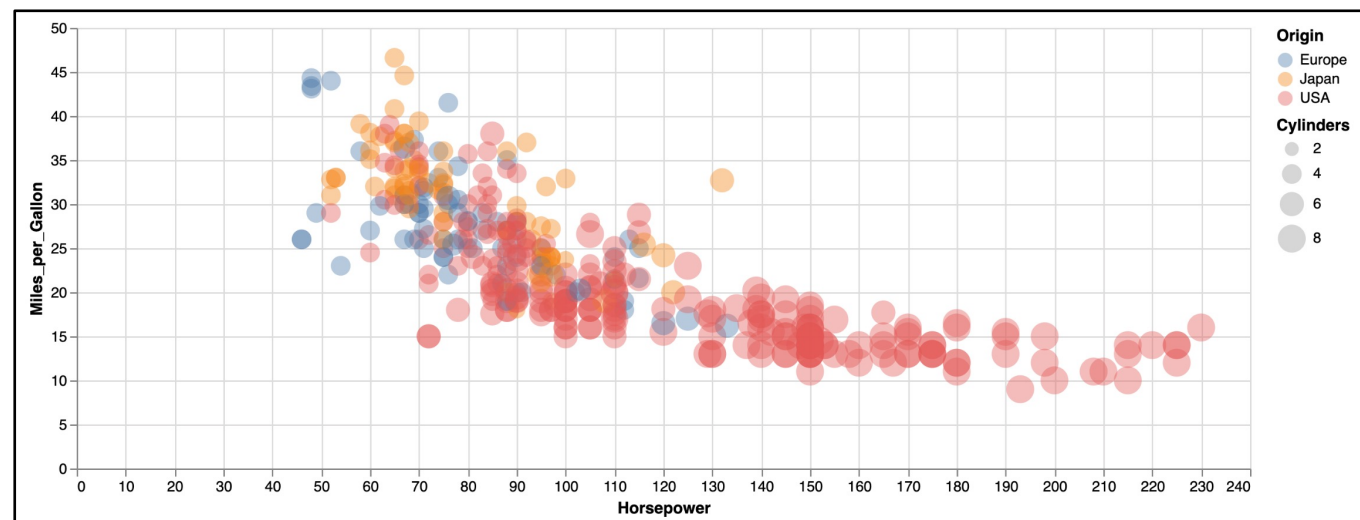
	Name	Miles_per_Gallon	Cylinders	Displacement	Horsepower	Weight_in_lbs	Acceleration	Year	Origin
0	chevrolet chevelle malibu	18.0	8	307.0	130.0	3504	12.0	1970-01-01	USA
1	buick skylark 320	15.0	8	350.0	165.0	3693	11.5	1970-01-01	USA
2	plymouth satellite	18.0	8	318.0	150.0	3436	11.0	1970-01-01	USA
3	amc rebel sst	16.0	8	304.0	150.0	3433	12.0	1970-01-01	USA
4	ford torino	17.0	8	302.0	140.0	3449	10.5	1970-01-01	USA
5	ford galaxie 500	15.0	8	429.0	198.0	4341	10.0	1970-01-01	USA
6	chevrolet impala	14.0	8	454.0	220.0	4354	9.0	1970-01-01	USA
7	plymouth fury iii	14.0	8	440.0	215.0	4312	8.5	1970-01-01	USA
8	pontiac catalina	14.0	8	455.0	225.0	4425	10.0	1970-01-01	USA
9	amc ambassador dpl	15.0	8	390.0	190.0	3850	8.5	1970-01-01	USA
10	citroen ds-21 pallas	NaN	4	133.0	115.0	3090	17.5	1970-01-01	Europe
11	chevrolet chevelle concours (sw)	NaN	8	350.0	165.0	4142	11.5	1970-01-01	USA
12	ford torino (sw)	NaN	8	351.0	153.0	4034	11.0	1970-01-01	USA
13	plymouth satellite (sw)	NaN	8	383.0	175.0	4166	10.5	1970-01-01	USA
14	amc rebel sst (sw)	NaN	8	360.0	175.0	3850	11.0	1970-01-01	USA
15	dodge challenger se	15.0	8	383.0	170.0	3563	10.0	1970-01-01	USA
16	plymouth 'cuda 340	14.0	8	340.0	160.0	3609	8.0	1970-01-01	USA
17	ford mustang boss 302	NaN	8	302.0	140.0	3353	8.0	1970-01-01	USA
18	chevrolet monte carlo	15.0	8	400.0	150.0	3761	9.5	1970-01-01	USA
19	buick estate wagon (sw)	14.0	8	455.0	225.0	3086	10.0	1970-01-01	USA

Map "data" to "visual elements"



What are we trying to do in a visualization?

	Name	Miles_per_Gallon	Cylinders	Displacement	Horsepower	Weight_in_lbs	Acceleration	Year	Origin
0	chevrolet chevelle malibu	18.0	8	307.0	130.0	3504	12.0	1970-01-01	USA
1	buick skylark 320	15.0	8	350.0	165.0	3693	11.5	1970-01-01	USA
2	plymouth satellite	18.0	8	318.0	150.0	3436	11.0	1970-01-01	USA
3	amc rebel sst	16.0	8	304.0	150.0	3433	12.0	1970-01-01	USA
4	ford torino	17.0	8	302.0	140.0	3449	10.5	1970-01-01	USA
5	ford galaxie 500	15.0	8	429.0	198.0	4341	10.0	1970-01-01	USA
6	chevrolet impala	14.0	8	454.0	220.0	4354	9.0	1970-01-01	USA
7	plymouth fury iii	14.0	8	440.0	215.0	4312	8.5	1970-01-01	USA
8	pontiac catalina	14.0	8	455.0	225.0	4425	10.0	1970-01-01	USA
9	amc ambassador dpl	15.0	8	390.0	190.0	3850	8.5	1970-01-01	USA
10	citroen ds-21 pallas	NaN	4	133.0	115.0	3090	17.5	1970-01-01	Europe
11	chevrolet chevelle concours (sw)	NaN	8	350.0	165.0	4142	11.5	1970-01-01	USA
12	ford torino (sw)	NaN	8	351.0	153.0	4034	11.0	1970-01-01	USA
13	plymouth satellite (sw)	NaN	8	383.0	175.0	4166	10.5	1970-01-01	USA
14	amc rebel sst (sw)	NaN	8	360.0	175.0	3850	11.0	1970-01-01	USA
15	dodge challenger se	15.0	8	383.0	170.0	3563	10.0	1970-01-01	USA
16	plymouth 'cuda 340	14.0	8	340.0	160.0	3609	8.0	1970-01-01	USA
17	ford mustang boss 302	NaN	8	302.0	140.0	3353	8.0	1970-01-01	USA
18	chevrolet monte carlo	15.0	8	400.0	150.0	3761	9.5	1970-01-01	USA
19	buick estate wagon (sw)	14.0	8	455.0	225.0	3086	10.0	1970-01-01	USA

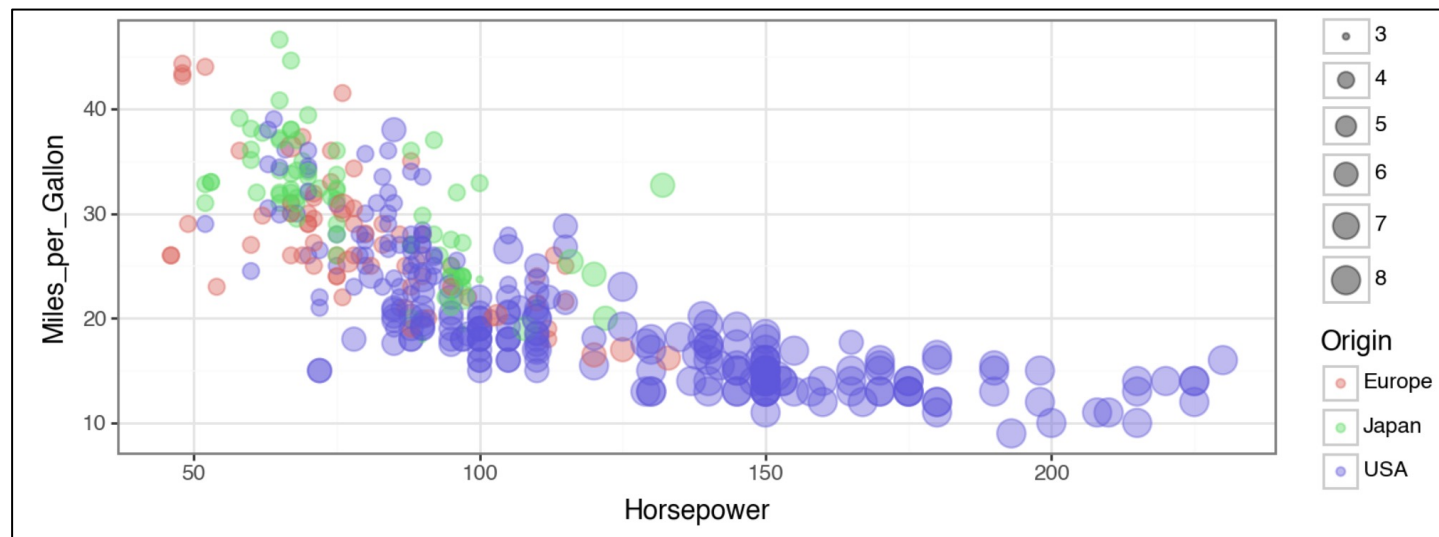


```
alt.Chart(cars).mark_circle(opacity = 0.4).encode(  
  x='Horsepower',  
  y='Miles_per_Gallon',  
  color='Origin',  
  size = 'Cylinders',  
)
```

Map data to visual elements

What are we trying to do in a visualization? Plotnine

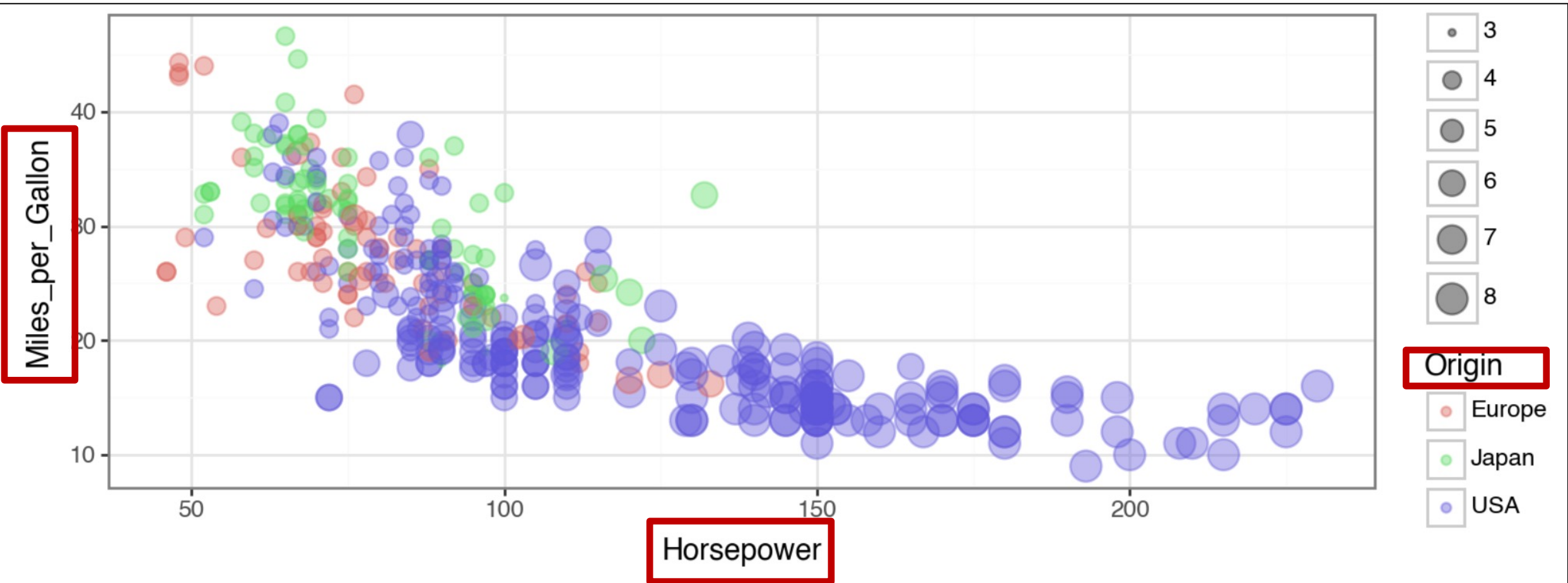
	Name	Miles_per_Gallon	Cylinders	Displacement	Horsepower	Weight_in_lbs	Acceleration	Year	Origin
0	chevrolet chevelle malibu	18.0	8	307.0	130.0	3504	12.0	1970-01-01	USA
1	buick skylark 320	15.0	8	350.0	165.0	3693	11.5	1970-01-01	USA
2	plymouth satellite	18.0	8	318.0	150.0	3436	11.0	1970-01-01	USA
3	amc rebel sst	16.0	8	304.0	150.0	3433	12.0	1970-01-01	USA
4	ford torino	17.0	8	302.0	140.0	3449	10.5	1970-01-01	USA
5	ford galaxie 500	15.0	8	429.0	198.0	4341	10.0	1970-01-01	USA
6	chevrolet impala	14.0	8	454.0	220.0	4354	9.0	1970-01-01	USA
7	plymouth fury iii	14.0	8	440.0	215.0	4312	8.5	1970-01-01	USA
8	pontiac catalina	14.0	8	455.0	225.0	4425	10.0	1970-01-01	USA
9	amc ambassador dpl	15.0	8	390.0	190.0	3850	8.5	1970-01-01	USA
10	citroen ds-21 pallas	NaN	4	133.0	115.0	3090	17.5	1970-01-01	Europe
11	chevrolet chevelle concours (sw)	NaN	8	350.0	165.0	4142	11.5	1970-01-01	USA
12	ford torino (sw)	NaN	8	351.0	153.0	4034	11.0	1970-01-01	USA
13	plymouth satellite (sw)	NaN	8	383.0	175.0	4166	10.5	1970-01-01	USA
14	amc rebel sst (sw)	NaN	8	360.0	175.0	3850	11.0	1970-01-01	USA
15	dodge challenger se	15.0	8	383.0	170.0	3563	10.0	1970-01-01	USA
16	plymouth 'cuda 340	14.0	8	340.0	160.0	3609	8.0	1970-01-01	USA
17	ford mustang boss 302	NaN	8	302.0	140.0	3353	8.0	1970-01-01	USA
18	chevrolet monte carlo	15.0	8	400.0	150.0	3761	9.5	1970-01-01	USA
19	buick estate wagon (sw)	14.0	8	455.0	225.0	3086	10.0	1970-01-01	USA



```
ggplot(cars)
  + aes(    x = 'Horsepower',
            y = 'Miles_per_Gallon',
            color = 'Origin')
  + geom_point(aes(size='Cylinders'), alpha=0.4)
  + theme_bw() + theme(figure_size=(8, 3))
```

Map data to visual elements

Map "data" to "visual elements"



Spend some time with these two papers (week 02a of the schedule)

How to Read a Paper

Version of February 17, 2016

S. Keshav

David R. Cheriton School of Computer Science, University of Waterloo
Waterloo, ON, Canada
keshav@uwaterloo.ca

ABSTRACT

Researchers spend a great deal of time reading research papers. However, this skill is rarely taught, leading to much wasted effort. This article outlines a practical and efficient *three-pass method* for reading research papers. I also describe how to use this method to do a literature survey.

1. INTRODUCTION

Researchers must read papers for several reasons: to review them for a conference or a class, to keep current in their field, or for a literature survey of a new field. A typical researcher will likely spend hundreds of hours every year reading papers.

Learning to efficiently read a paper is a critical but rarely taught skill. Beginning graduate students, therefore, must learn on their own using trial and error. Students waste much effort in the process and are frequently driven to frustration.

For many years I have used a simple ‘three-pass’ approach to prevent me from drowning in the details of a paper before getting a bird’s-eye-view. It allows me to estimate the amount of time required to review a set of papers. Moreover, I can adjust the depth of paper evaluation depending on my needs and how much time I have. This paper describes the approach and its use in doing a literature survey.

2. THE THREE-PASS APPROACH

The key idea is that you should read the paper in up to three passes, instead of starting at the beginning and plowing your way to the end. Each pass accomplishes specific goals and builds upon the previous pass: The *first* pass gives you a general idea about the paper. The *second* pass lets you grasp the paper’s content, but not its details. The *third* pass helps you understand the paper in depth.

2.1 The first pass

4. Read the conclusions

5. Glance over the references, mentally ticking off the ones you’ve already read

At the end of the first pass, you should be able to answer the *five Cs*:

1. *Category*: What type of paper is this? A measurement paper? An analysis of an existing system? A description of a research prototype?

2. *Context*: Which other papers is it related to? Which theoretical bases were used to analyze the problem?

3. *Correctness*: Do the assumptions appear to be valid?

4. *Contributions*: What are the paper’s main contributions?

5. *Clarity*: Is the paper well written?

Using this information, you may choose not to read further (and not print it out, thus saving trees). This could be because the paper doesn’t interest you, or you don’t know enough about the area to understand the paper, or that the authors make invalid assumptions. The first pass is adequate for papers that aren’t in your research area, but may someday prove relevant.

Incidentally, when you write a paper, you can expect most reviewers (and readers) to make only one pass over it. Take care to choose coherent section and sub-section titles and to write concise and comprehensive abstracts. If a reviewer cannot understand the gist after one pass, the paper will likely be rejected; if a reader cannot understand the highlights of the paper after five minutes, the paper will likely never be read. For these reasons, a ‘graphical abstract’ that summarizes a paper with a single well-chosen figure is an excellent idea and can be increasingly found in scientific journals.



Journal of Statistical Software

MMMMMM YYYY, Volume VV, Issue II.

<http://www.jstatsoft.org/>

Tidy Data

Hadley Wickham
RStudio

Abstract

A huge amount of effort is spent cleaning data to get it ready for analysis, but there has been little research on how to make data cleaning as easy and effective as possible. This paper tackles a small, but important, component of data cleaning: data tidying. Tidy datasets are easy to manipulate, model and visualise, and have a specific structure: each variable is a column, each observation is a row, and each type of observational unit is a table. This framework makes it easy to tidy messy datasets because only a small set of tools are needed to deal with a wide range of un-tidy datasets. This structure also makes it easier to develop tidy tools for data analysis, tools that both input and output tidy datasets. The advantages of a consistent data structure and matching tools are demonstrated with a case study free from mundane data manipulation chores.

Keywords: data cleaning, data tidying, relational databases, R.

TIDY DATA

Variables, Values, and Observations

The next five slides in this section are by Garret Grolemund

Do you notice a pattern i.e., a relationship between the values of X , Y, and Z?

X	Y	Z
3.01	0.98	3.08
2.35	0.91	2.58
5.57	1.01	5.52
0.62	1.09	0.56
4.15	0.89	4.69
5.07	1.05	4.84
7.56	0.93	8.12

Variables, Values, and Observations

$$\mathbf{F} = \mathbf{M} \mathbf{A}$$

$$\begin{array}{ccc} f_1 & m_1 & a_1 \\ f_2 & m_2 & a_2 \\ f_3 & m_3 & a_3 \end{array}$$

Variable - A quantity, quality, or property that you can measure

Value - The state of a variable when you measure it (data)

Observation - The values of several variables measured under similar conditions.

Natural Laws

$$\mathbf{F} = \mathbf{M} \mathbf{A}$$

$$\begin{array}{ccc} f_1 & m_1 & a_1 \\ f_2 & m_2 & a_2 \\ f_3 & m_3 & a_3 \end{array}$$

Structure of Natural Laws

Natural laws deal with **variables**,
but they operate on **values**
that appear in the same **observation**.

Patterns

F	M	A
3.01	0.98	3.08
2.35	0.91	2.58
5.57	1.01	5.52
0.62	1.09	0.56
4.15	0.89	4.69
5.07	1.05	4.84
7.56	0.93	8.12

$$f_1 = m_1 \cdot a_1$$

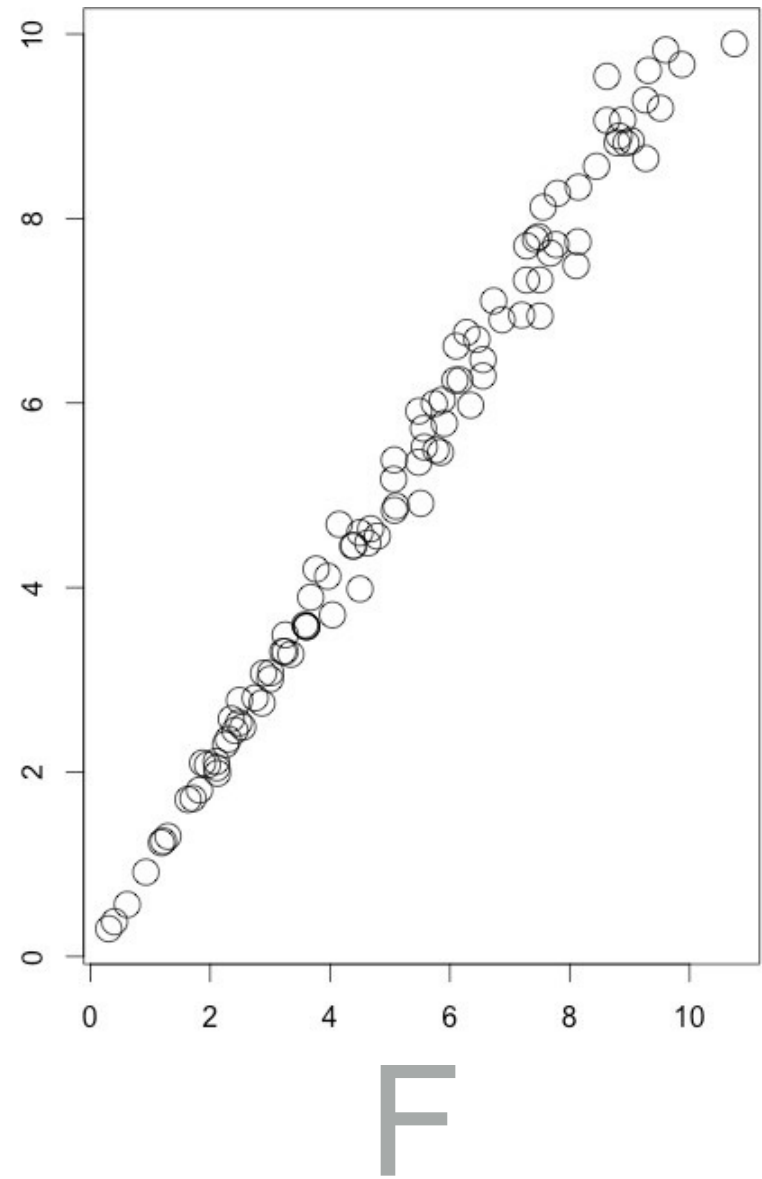
$$f_2 = m_2 \cdot a_2$$

$$f_3 = m_3 \cdot a_3$$

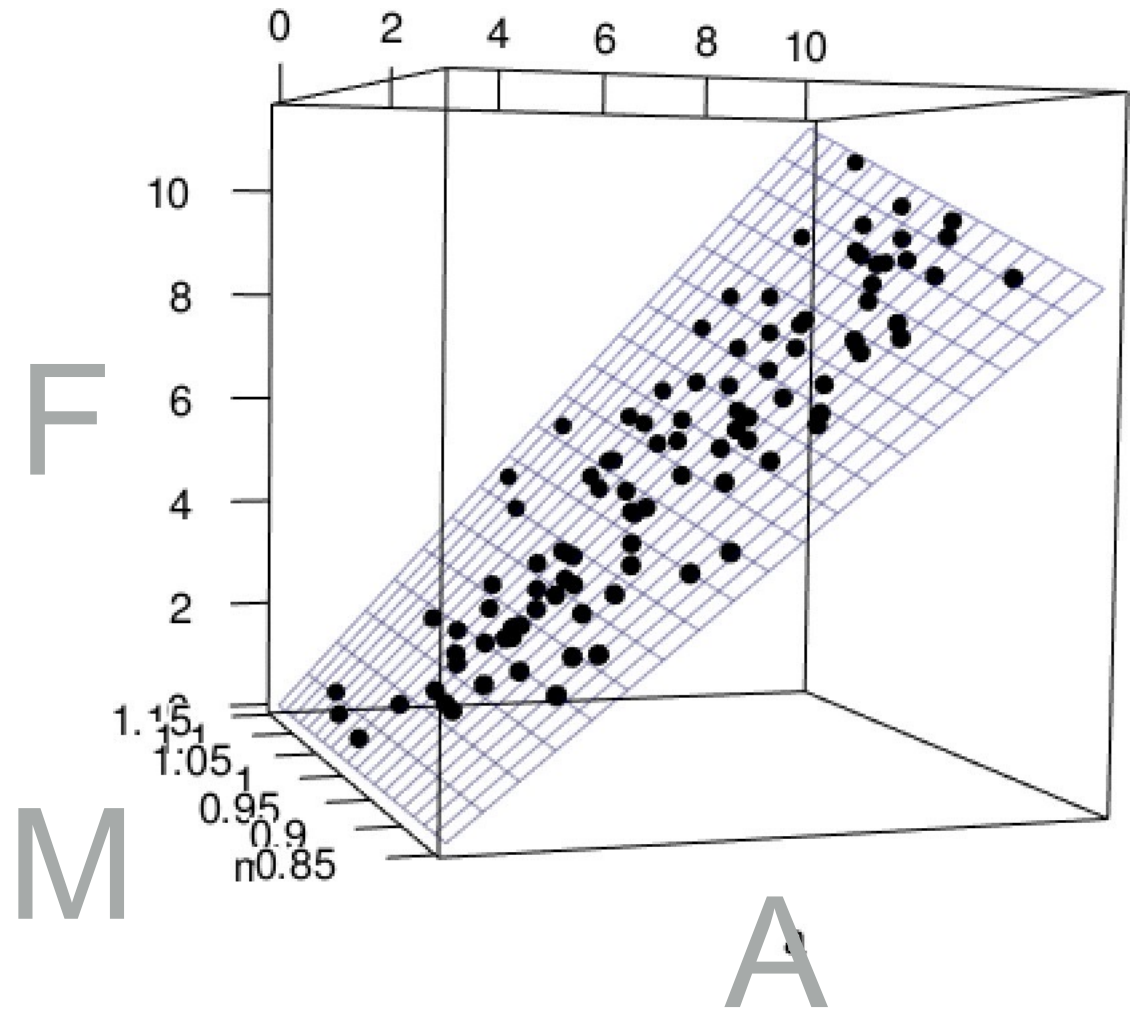
Laws appear as
patterns in data.

F	M	A
0.62	1.09	0.56
1.30	0.99	1.30
1.63	0.96	1.70
1.72	1.00	1.71
1.82	1.01	1.80
1.95	0.94	2.08
2.11	1.03	2.05

A

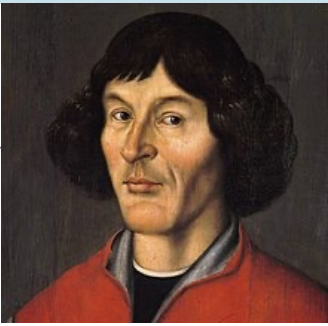


F	M	A
0.62	1.09	0.56
1.30	0.99	1.30
1.63	0.96	1.70
1.72	1.00	1.71
1.82	1.01	1.80
1.95	0.94	2.08
2.11	1.03	2.05

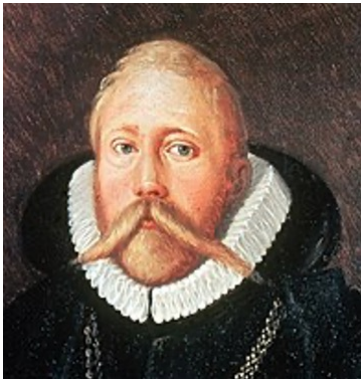


EARLY DATA SCIENCE

Patterns from Data



Nicolas Copernicus (1473—1543)



Tycho Brahe (1546 – 1601)



Johannes Kepler (1571 – 1630)



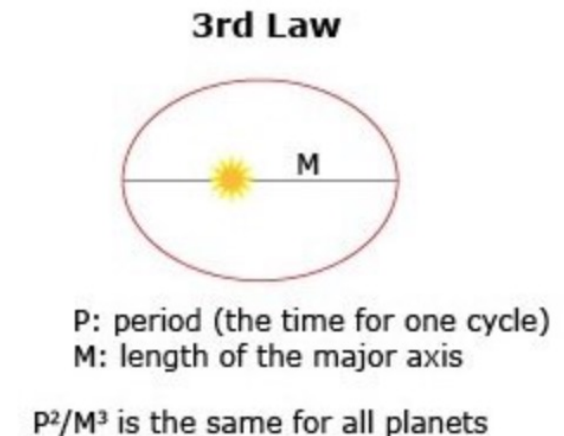
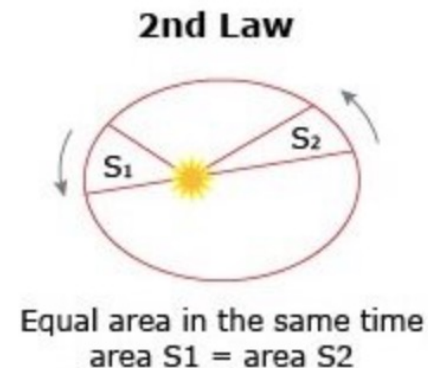
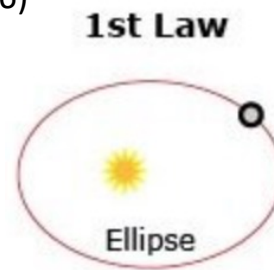
Galileo Galilei (1564 – 1642)



Issac Newton (1642 – 1726)

Kepler's Laws of Planetary Motion

1. The orbit of a planet is an ellipse with the Sun at one of the two foci.
2. A line segment joining a planet and the Sun sweeps out equal areas during equal intervals of time.
3. The square of a planet's orbital period is proportional to the cube of the length of the semi-major axis of its orbit.



Guess, then verify

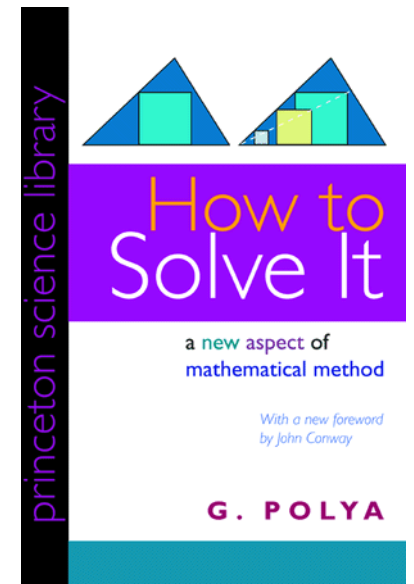
How Did Kepler Discover His First Two Laws?

It is generally assumed that he did so by calculating the distances between a planet and the sun and then perceiving that the distances fitted into an ellipse. It is more likely that the ellipse came first

Curtis Wilson, Scientific American, -1972-03-01, Vol.226 (3), p.92-107

Finished mathematics is about theorems;
Mathematics in the making is about guesses
... George Polya

"the greatest piece of
retroductive reasoning ever
performed."
... Charles Sanders Peirce
(19th century philosopher of
Science)



1 800 DISCOVER
VS
1 800 347 2683

TIDY DATA

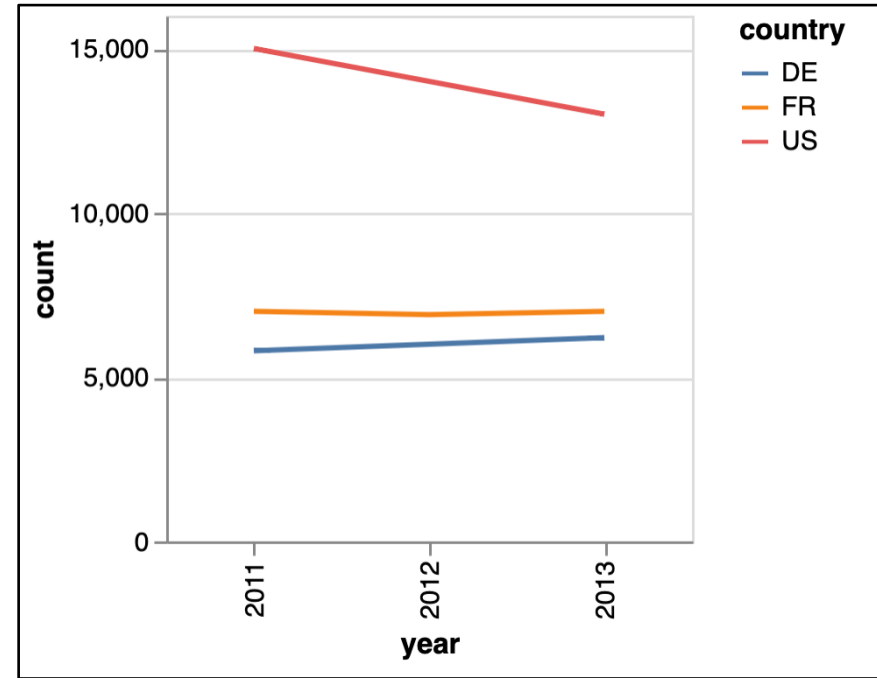
Form follows function

Mapping *variables*, *observations*, and *types* to *Columns*, *Rows*, and *Tables*

Two uses of Data ... Visualization and Prediction

countries				
	country	2011	2012	2013
0	FR	7000	6900	7000
1	DE	5800	6000	6200
2	US	15000	14000	13000

visualize



estimate

Predict:
US / 2014 ?

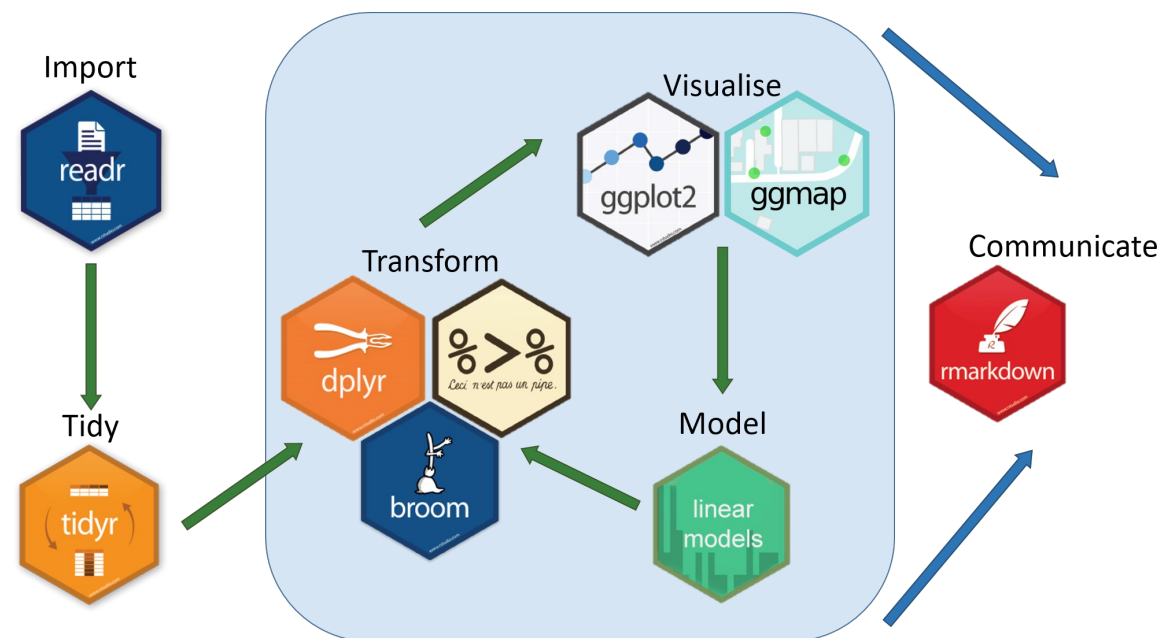
Definition of Tidy Data



1. Each **variable** forms a **column**
2. Each **observation** forms a **row**
3. Each type of **observational unit** forms a **table**



Hadley Wickham



Example 1: What are the variables?

Storm	Wind	Pressue	Date
Alberto	110	1007	2000-08-12
Alex	45	1009	1998-07-30
Allison	65	1005	1995-06-04
Ana	40	1013	1997-07-01
Arlene	50	1010	1996-06-13
Arthur	45	1010	1996-06-21

Variable - A quantity, quality, or property that you can measure

Value - The state of a variable when you measure it

Observation - The values of several variables measured under similar conditions.

Example 1: Tidy form

Storm	Wind	Pressure	Date
Alberto	110	1007	2000-08-12
Alex	45	1009	1998-07-30
Allison	65	1005	1995-06-04
Ana	40	1013	1997-07-01
Arlene	50	1010	1996-06-13
Arthur	45	1010	1996-06-21

- Storm name
 - Wind (speed, mph)
 - Air Pressure (millibars)
 - Date
- (per storm)

Example 2: What are the variables?

Country	2011	2012	2013
FR	7000	6900	7000
DE	5800	6000	6200
US	15000	14000	13000

Variable - A quantity, quality, or property that you can measure

Value - The state of a variable when you measure it

Observation - The values of several variables measured under similar conditions

Example 2: Tidy form

Country	2011	2012	2013
FR	7000	6900	7000
DE	5800	6000	6200
US	15000	14000	13000

- Country
 - Year
 - Count
- (per country per year)

Country	Year	Count
FR	2011	7000
DE	2011	5800
US	2011	15000
FR	2012	6900
DE	2012	6000
US	2012	14000
FR	2013	7000
DE	2013	6200
US	2013	13000

Extracting Variables and Observations are easier in Tidy Data

Country	2011	2012	2013
FR	7000	6900	7000
DE	5800	6000	6200
US	15000	14000	13000

messy

```
country =  
year =  
count =
```

```
country =  
year =  
count =
```

Country	Year	Count
FR	2011	7000
DE	2011	5800
US	2011	15000
FR	2012	6900
DE	2012	6000
US	2012	14000
FR	2013	7000
DE	2013	6200
US	2013	13000

tidy

Extracting Variables and Observations are easier in Tidy Data

Country	2011	2012	2013
FR	7000	6900	7000
DE	5800	6000	6200
US	15000	14000	13000

messy

```
country = df.country
year = df.columns[1:]
count = df.values
```

```
country = df.country
year = df.year
count = df.count
```

Country	Year	Count
FR	2011	7000
DE	2011	5800
US	2011	15000
FR	2012	6900
DE	2012	6000
US	2012	14000
FR	2013	7000
DE	2013	6200
US	2013	13000

tidy

Example 3: What are the variables?

City	Particle size	Amount ($\mu\text{g}/\text{m}^3$)
New York	large	23
New York	small	14
London	large	22
London	small	16
Beijing	large	121
Beijing	small	56

Variable - A quantity, quality, or property that you can measure

Value - The state of a variable when you measure it

Observation - The values of several variables measured under similar conditions

Example 3: Tidy form

City	Particle size	Amount (µg/m³)
New York	large	23
New York	small	14
London	large	22
London	small	16
Beijing	large	121
Beijing	small	56

City	Large	small
New York	23	14
London	22	16
Beijing	121	56

- City
- Amount of large particles
- Amount of small particles

(per city)

RESHAPING DATAFRAMES

Melt

Pivot

Pivot_table

Unstack

Stack

pd.melt

countries				
	country	2011	2012	2013
0	FR	7000	6900	7000
1	DE	5800	6000	6200
2	US	15000	14000	13000

```
pd.melt(countries,  
        id_vars=['country'],  
        value_vars=['2011', '2012', '2013'],  
        var_name='year',  
        value_name='count')
```

	country	year	count
0	FR	2011	7000
1	DE	2011	5800
2	US	2011	15000
3	FR	2012	6900
4	DE	2012	6000
5	US	2012	14000
6	FR	2013	7000
7	DE	2013	6200
8	US	2013	13000

pd.melt

dataframe

Existing variables
already in columns

Values of the new
variable

Name of the new
variable

Name of the new
value column

```
pd.melt(countries,  
        id_vars=['country'],  
        value_vars=['2011', '2012', '2013'],  
        var_name='year',  
        value_name='count')
```

countries				
	country	2011	2012	2013
0	FR	7000	6900	7000
1	DE	5800	6000	6200
2	US	15000	14000	13000

Five Reasons for Untidyness

1. Column headers are values, not variable names
2. Multiple variables are stored in one column
3. Variables are stored in both rows and columns
4. Multiple types of observational units are stored in the same table
5. A single observational unit is stored in multiple tables

Happy families are all alike;
every unhappy family is unhappy in its own way
— Leo Tolstoy

Full version of the data

	religion	<\$10k	\$10-20k	\$20-30k	\$30-40k	\$40-50k	\$50-75k	\$75-100k	\$100-150k	>150k	Don't know/refused
0	Agnostic	27	34	60	81	76	137	122	109	84	96
1	Atheist	12	27	37	52	35	70	73	59	74	76
2	Buddhist	27	21	30	34	33	58	62	39	53	54
3	Catholic	418	617	732	670	638	1116	949	792	633	1489
4	Don't know/refused	15	14	15	11	10	35	21	17	18	116
5	Evangelical Prot	575	869	1064	982	881	1486	949	723	414	1529
6	Hindu	1	9	7	9	11	34	47	48	54	37
7	Historically Black Prot	228	244	236	238	197	223	131	81	78	339
8	Jehovah's Witness	20	27	24	24	21	30	15	11	6	37
9	Jewish	19	19	25	25	30	95	69	87	151	162
10	Mainline Prot	289	495	619	655	651	1107	939	753	634	1328
11	Mormon	29	40	48	51	56	112	85	49	42	69
12	Muslim	6	7	9	10	9	23	16	8	6	22
13	Orthodox	13	17	23	32	32	47	38	42	46	73
14	Other Christian	9	7	11	13	13	14	18	14	12	18
15	Other Faiths	20	33	40	46	49	63	46	40	41	71
16	Other World Religions	5	2	3	4	2	7	3	4	4	8
17	Unaffiliated	217	299	374	365	341	528	407	321	258	597

1. Column headers are values, not variable names 1/4

	<\$10k	\$10-20k	\$20-30k	\$30-40k	\$40-50k	\$50-75k
religion						
Agnostic	27	34	60	81	76	137
Atheist	12	27	37	52	35	70
Buddhist	27	21	30	34	33	58
Catholic	418	617	732	670	638	1116
Dont know/refused	15	14	15	11	10	35
Evangelical Prot	575	869	1064	982	881	1486
Hindu	1	9	7	9	11	34
Historically Black Prot	228	244	236	238	197	223
Jehovahs Witness	20	27	24	24	21	30
Jewish	19	19	25	25	30	95

What are the variables?

What are the observations?

1. Column headers are values, not variable names 2/4

		<\$10k	\$10-20k	\$20-30k	\$30-40k	\$40-50k	\$50-75k
religion							
Agnostic		27	34	60	81	76	137
Atheist		12	27	37	52	35	70
Buddhist		27	21	30	34	33	58
Catholic		418	617	732	670	638	1116
Dont know/refused		15	14	15	11	10	35
Evangelical Prot		575	869	1064	982	881	1486
Hindu		1	9	7	9	11	34
Historically Black Prot		228	244	236	238	197	223
Jehovahs Witness		20	27	24	24	21	30
Jewish		19	19	25	25	30	95

What are the variables?

1. Column headers are values, not variable names 3/4

	<\$10k	\$10-20k	\$20-30k	\$30-40k	\$40-50k	\$50-75k
religion						
Agnostic	27	34	60	81	76	137
Atheist	12	27	37	52	35	70
Buddhist	27	21	30	34	33	58
Catholic	418	617	732	670	638	1116
Dont know/refused	15	14	15	11	10	35
Evangelical Prot	575	869	1064	982	881	1486
Hindu	1	9	7	9	11	34
Historically Black Prot	228	244	236	238	197	223
Jehovahs Witness	20	27	24	24	21	30
Jewish	19	19	25	25	30	95

What are the observations?

1. Column headers are values, not variable names 4/4

10 x 6

		<\$10k	\$10-20k	\$20-30k	\$30-40k	\$40-50k	\$50-75k
religion							
Agnostic		27	34	60	81	76	137
Atheist		12	27	37	52	35	70
Buddhist		27	21	30	34	33	58
Catholic		418	617	732	670	638	1116
Dont know/refused		15	14	15	11	10	35
Evangelical Prot		575	869	1064	982	881	1486
Hindu		1	9	7	9	11	34
Historically Black Prot		228	244	236	238	197	223
Jehovahs Witness		20	27	24	24	21	30
Jewish		19	19	25	25	30	95

60 x 3

religion	income	freq
Agnostic	<\$10k	27
Agnostic	\$30-40k	81
Agnostic	\$40-50k	76
Agnostic	\$50-75k	137
Agnostic	\$10-20k	34
Agnostic	\$20-30k	60
Atheist	\$40-50k	35
Atheist	\$20-30k	37
Atheist	\$10-20k	27
Atheist	\$30-40k	52

pd.pivot / pd.pivot_table

	e1	e2	e3
Bob			
Jack			
Jill			

	e1	e2	e3
Bob	97	-	99
Jack	92, 98	83	-
Jill	96	-	86 97

	Name	Exam	Score
0	Jack	e1	92
1	Jill	e1	96
2	Jack	e2	83
3	Bob	e1	97
4	Jill	e3	86
5	Jack	e1	98
6	Bob	e3	99
7	Jill	e3	97

← columns

```
pd.pivot_table(  
    df,  
    index='name',  
    columns='exam',  
    values='score',  
    aggfunc = lambda s: s.mean()  
)
```

)

0	92
5	98

↑
index

	e1	e2	e3
Bob	97	NaN	99
Jack	95	83	NaN
Jill	96	NaN	91.5

pd.stack / unstack

```
df.unstack()  
df.unstack(-1)  
df.unstack(1)
```

		SAT	ACT
PA	Jack	1440	33
PA	Jill	1550	36
FL	Bob	1340	30
IN	Harry	1440	34
IN	Pat	1530	35
IN	Carol	1490	31

	sat						act					
name	Bob	Carol	Harry	Jack	Jill	Pat	Bob	Carol	Harry	Jack	Jill	Pat
state												
FL	1340	NaN	NaN	NaN	NaN	NaN	30	NaN	NaN	NaN	NaN	NaN
IN	NaN	1490	1440	NaN	NaN	1530	NaN	31	34	NaN	NaN	35
PA	NaN	NaN	NaN	1440	1550	NaN	NaN	NaN	NaN	33	36	NaN

```
df.unstack(0)
```

	sat			act		
state	FL	IN	PA	FL	IN	PA
name						
Bob	1340	NaN	NaN	30	NaN	NaN
Carol	NaN	1490	NaN	NaN	31	NaN
Harry	NaN	1440	NaN	NaN	34	NaN
Jack	NaN	NaN	1440	NaN	NaN	33
Jill	NaN	NaN	1550	NaN	NaN	36
Pat	NaN	1530	NaN	NaN	35	NaN